

## Screenshot Program Python 1 – 6

### #program 1

```
def hitung_kata_dan_kalimat(teks):
    daftar_kata = teks.split()
    jumlah_kata = len(daftar_kata)

    daftar_kalimat = teks.split(".")
    jumlah_kalimat = 0

    for kalimat in daftar_kalimat:
        if kalimat.strip() != "":
            jumlah_kalimat += 1

    print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")

# Contoh pemanggilan fungsi
hitung_kata_dan_kalimat("Ketuhanan Yang Maha Esa. Kemanusiaan yang adil dan beradab. Persatuan Indonesia.")
hitung_kata_dan_kalimat("Fungsi adalah blok kode yang diberi nama dan digunakan untuk menjalankan tugas tertentu. Fungsi dapat menerima input berupa parameter atau argumen, memproses input tersebut, kemudian menghasilkan output.")
hitung_kata_dan_kalimat("aku berlari menjauh dari hal yang tak lagi utuh. tetes air mata serupa hujan basahiku sepanjang jalan. dalam lelah kuberharap agar tuhan berikan aku lupa. namun itu tak kudapatkan kisah lain yang ia berikan.")

***
Teks tersebut memuat 3 kalimat dan 11 kata.
Teks tersebut memuat 2 kalimat dan 27 kata.
Teks tersebut memuat 4 kalimat dan 34 kata.
```

### #program 2

```
#Soal 2 Membuat Objek String K1 sampai K4
K1="Saya tak 'kan menyerah."
K2="Ia berkata, "Aku menyayangimu.""
K3="\nCoba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
K4="Surat keputusan itu bernomor 62/UN.34/19/2023."

print(K1)
print(K2)
print(K3)
print(K4)

Saya tak 'kan menyerah.
Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.
```

### #program 3

```
import math

def hitung_akar_kuadrat(koefisien_a, koefisien_b, koefisien_c):
    diskriminan = (koefisien_b**2) - (4 * koefisien_a * koefisien_c)

    if diskriminan > 0:
        akar_1 = (-koefisien_b + math.sqrt(diskriminan)) / (2 * koefisien_a)
        akar_2 = (-koefisien_b - math.sqrt(diskriminan)) / (2 * koefisien_a)

        print("Persamaan tersebut memiliki akar real:")
        print(f"Akar 1 = {akar_1:.3f}")
        print(f"Akar 2 = {akar_2:.3f}\n")

    elif diskriminan == 0:
        akar = -koefisien_b / (2 * koefisien_a)

        print("Persamaan tersebut memiliki akar real kembar:")
        print(f"Akar = {akar:.3f}\n")

    else:
        print("Persamaan tersebut hanya memiliki akar-akar imajiner\n")

#Contoh pemanggilan fungsi
hitung_akar_kuadrat(1, -4, 4)
hitung_akar_kuadrat(2, -13, 15)
hitung_akar_kuadrat(3, -10, 25)

***
Persamaan tersebut memiliki akar real kembar:
Akar = 2.000

Persamaan tersebut memiliki akar real:
Akar 1 = 5.000
Akar 2 = 1.500

Persamaan tersebut hanya memiliki akar-akar imajiner
```

## #program 4

```
[3] ✓ Os def tanggal_lahir(nip):
nip_str = str(nip)

tahun = int(nip_str[0:4])
bulan = int(nip_str[4:6])
tanggal = int(nip_str[6:8])

daftar_bulan = [
    "Januari",
    "Februari",
    "Maret",
    "April",
    "Mei",
    "Juni",
    "Juli",
    "Agustus",
    "September",
    "Oktober",
    "November",
    "Desember",
]

if 1 <= bulan <= 12:
    nama_bulan = daftar_bulan[bulan - 1]
    print(f"Tanggal lahir ASN: (tanggal) {nama_bulan} {tahun}\n")
else:
    print("Bulan Tidak Valid\n")

#Contoh pemanggilan fungsi
tanggal_lahir(199210252015032015)
tanggal_lahir(198540122010011003)
tanggal_lahir(199812012023021001)

Tanggal lahir ASN: 25 Oktober 1992

Bulan Tidak Valid

Tanggal lahir ASN: 1 Desember 1998
```

## #program 5

```
import numpy as np
def cluster_titik(x1, x2, x3):
    A = np.array([3, 2, 4])
    B = np.array([-2, 5, 1])
    C = np.array([1, -3, -2])

    U = np.array([x1, x2, x3])

    dA = np.sqrt(np.sum((U - A) ** 2))
    dB = np.sqrt(np.sum((U - B) ** 2))
    dC = np.sqrt(np.sum((U - C) ** 2))

    print(f"Jarak ke A: {dA:.4f}")
    print(f"Jarak ke B: {dB:.4f}")
    print(f"Jarak ke C: {dC:.4f}")

    jarak_min = min(dA, dB, dC)
    cluster_terpilih = []

    if dA == jarak_min:
        cluster_terpilih.append("Cluster A")
    if dB == jarak_min:
        cluster_terpilih.append("Cluster B")
    if dC == jarak_min:
        cluster_terpilih.append("Cluster C")

    output_cluster = " dan ".join(cluster_terpilih)
    print(f"Titik termasuk {output_cluster}\n")

#Contoh pemanggilan fungsi
cluster_titik(4, 3, 7)
cluster_titik(3, -7, -1)
cluster_titik(0.5, 3.5, 2.5)

''' Jarak ke A: 3.3166
Jarak ke B: 8.7178
Jarak ke C: 11.2250
Titik termasuk Cluster A

Jarak ke A: 10.2956
Jarak ke B: 13.1529
Jarak ke C: 4.5826
Titik termasuk Cluster C

Jarak ke A: 3.2787
Jarak ke B: 3.2787
Jarak ke C: 7.9235
Titik termasuk Cluster A dan Cluster B
```

## #program 6

```
import math
def interval_konfidensi(p_hat, n, alpha):
    if p_hat < 0 or p_hat > 1:
        print("Error: Proporsi harus berada pada rentang 0 sampai 1.\n")
        return

    if alpha == 0.05:
        z = 1.96
    elif alpha == 0.10:
        z = 1.645
    else:
        print("Error: Alpha harus bernilai 0.05 atau 0.10.\n")
        return

    margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)

    batas_bawah = p_hat - margin_error
    batas_atas = p_hat + margin_error

    print(f"Proporsi Sampel = {p_hat}")
    print(f"Ukuran Sampel = {n}")
    print(f"Alpha = {alpha}")
    print(f"Interval Konfidensi = ( {batas_bawah:.3f} , {batas_atas:.3f} )\n")

#Contoh pemanggilan fungsi
interval_konfidensi(0.4, 100, 0.05)
interval_konfidensi(0.6, 250, 0.01)
interval_konfidensi(1.2, 150, 0.10)
```

```
Proporsi Sampel = 0.4
Ukuran Sampel = 100
Alpha = 0.05
Interval Konfidensi = ( 0.304 , 0.496 )

Error: Alpha harus bernilai 0.05 atau 0.10.

Error: Proporsi harus berada pada rentang 0 sampai 1.
```